

OOP Coding Practice Questions

Coding questions only — input/test cases and expected output. No solutions or hints.

Level 1 — Objects & Instance Variables

1. Student Details

Create a **Student** class with instance variables: **name**, **rollNo**, **marks**. Create student objects and display their details.

Test Case 1

Input:

Uday 101 85

Ravi 102 92

Expected Output:

Name: Uday, Roll No: 101, Marks: 85

Name: Ravi, Roll No: 102, Marks: 92

Test Case 2

Input:

Anil 201 76

Priya 202 89

Expected Output:

Name: Anil, Roll No: 201, Marks: 76

Name: Priya, Roll No: 202, Marks: 89

Level 2 — Instance Methods

2. Bank Account

Create a **BankAccount** class containing: account number, account holder name, balance. Create methods: `deposit()`, `withdraw()`, `displayBalance()`. Each account should maintain its own balance.

Test Case 1

Input:

Account: 1001

Name: Uday

Initial Balance: 50000

Deposit: 5000

Withdraw: 10000

Expected Output:

Account Holder: Uday

Balance: 45000

Test Case 2

Input:

Account: 1002

Name: Ravi

Initial Balance: 30000
Deposit: 10000
Withdraw: 5000

Expected Output:

Account Holder: Ravi
Balance: 35000

Level 3 — Instance Variables + Methods

3. Employee Salary

Create an **Employee** class: name, salary. Create an instance method: increaseSalary(int percentage). The method should increase the salary of that particular employee.

Test Case 1

Input:

Name: Uday
Salary: 50000
Increase: 10%

Expected Output:

Name: Uday
Salary: 55000

Test Case 2

Input:

Name: Ravi
Salary: 40000
Increase: 20%

Expected Output:

Name: Ravi
Salary: 48000

Level 4 — Static Variable

4. Employee Counter

Create an **Employee** class. Every time an Employee object is created, increase a counter. Create: displayEmployeeCount(). The count should represent the total number of Employee objects created.

Test Case 1

Input:

Create:
Employee 1
Employee 2
Employee 3

Expected Output:

Total Employees: 3

Test Case 2

Input:

Create:

Employee 1

Employee 2

Employee 3

Employee 4

Employee 5

Expected Output:

Total Employees: 5

Level 5 — Static Variable + Instance Variable

5. Company Employees

Create an **Employee** class containing: name, salary, companyName, totalEmployees. name and salary should belong to individual employees. The company name and employee count should be shared.

Test Case

Input:

Company: ABC Technologies

Employee 1:

Uday 50000

Employee 2:

Ravi 60000

Employee 3:

Anil 45000

Expected Output:

Company: ABC Technologies

Uday - 50000

Ravi - 60000

Anil - 45000

Total Employees: 3

Level 6 — Static Method

6. Calculator

Create a **Calculator** class with static methods: add(), subtract(), multiply(), divide(). The methods should be callable using the class name.

Test Case 1

Input:

10 20

Expected Output:

Addition: 30

Subtraction: -10
Multiplication: 200
Division: 0.5

Test Case 2

Input:

50 10

Expected Output:

Addition: 60
Subtraction: 40
Multiplication: 500
Division: 5

Level 7 — Method Overloading

7. Area Calculator

Create an **AreaCalculator** class with overloaded methods: `area(side)`, `area(length, breadth)`, `area(radius)`.
Calculate: Square, Rectangle, Circle. Use $\pi = 3.14$

Test Case

Input:

Square side: 5
Rectangle: 10 20
Circle radius: 7

Expected Output:

Square Area: 25
Rectangle Area: 200
Circle Area: 153.94

Level 8 — Method Overloading

8. Print Details

Create overloaded methods: `display(String name)`, `display(String name, int age)`, `display(String name, int age, String city)`.

Test Case 1

Input:

Uday

Expected Output:

Name: Uday

Test Case 2

Input:

Uday 28

Expected Output:

Name: Uday
Age: 28

Test Case 3

Input:

Uday 28 Rajahmundry

Expected Output:

Name: Uday

Age: 28

City: Rajahmundry

Level 9 — Method Overloading

9. Food Order

Create overloaded methods: `order(String food)`, `order(String food, int quantity)`, `order(String food, int quantity, String address)`.

Test Case 1

Input:

Pizza

Expected Output:

Food: Pizza

Quantity: 1

Test Case 2

Input:

Burger 3

Expected Output:

Food: Burger

Quantity: 3

Test Case 3

Input:

Pizza 2 Rajahmundry

Expected Output:

Food: Pizza

Quantity: 2

Address: Rajahmundry

Level 10 — Method Overriding

10. Vehicle

Create a parent class: Vehicle, with start(). Create child classes: Car, Bike, ElectricBike. Each child should override start().

Test Case 1

Input:

Car

Expected Output:

Car engine starts

Test Case 2

Input:

Bike

Expected Output:

Bike engine starts

Test Case 3

Input:

ElectricBike

Expected Output:

Electric motor starts

Level 11 — Overriding + Parent Reference

11. Payment System

Create: Payment, CreditCardPayment, UPIPayment, CashPayment. Parent: pay(). Override pay() in every child. Use a parent-class reference to call the methods.

Test Case 1

Input:

CreditCard

Expected Output:

Payment using Credit Card

Test Case 2

Input:

UPI

Expected Output:

Payment using UPI

Test Case 3

Input:

Cash

Expected Output:

Payment using Cash

Level 12 — Overloading + Overriding

12. Payment Application

Create a parent class: Payment. Create overloaded methods: pay(int amount), pay(int amount, String currency). Create child classes: UPIPayment, CreditCardPayment. Override both methods.

Test Case 1

Input:

UPI
500

Expected Output:

UPI Payment: 500

Test Case 2

Input:

UPI
500 INR

Expected Output:

UPI Payment: 500 INR

Test Case 3

Input:

CreditCard
1000

Expected Output:

Credit Card Payment: 1000

Level 13 — Static + Instance + Overriding

13. Ride Booking

Create: Ride, BikeRide, AutoRide, CarRide. Ride should have: customerName, distance, totalRides. Create: calculateFare(). Each child should calculate differently: Bike → ■10/km, Auto → ■15/km, Car → ■20/km.

Test Case

Input:

Bike Uday 10
Auto Ravi 10
Car Anil 10

Expected Output:

Uday Bike Fare: 100
Ravi Auto Fare: 150
Anil Car Fare: 200

Total Rides: 3

Level 14 — Complete Challenge

14. EV Management System

Create: EV, ElectricBike, ElectricScooter, ElectricCar. EV should have: brand, model, batteryCapacity, price, companyName, totalVehicles. Implement: displayDetails(), calculateRange(), charge(). Use method overloading for charge(). Use method overriding for calculateRange(). Use a static variable to count all EVs.

Test Case

Input:

ElectricBike

Ola

Roadster

4

120000

ElectricScooter

Ather

450X

5

150000

ElectricCar

Tata

Nexon

40

1500000

Expected Output:

Company: ABC EV Motors

Electric Bike

Brand: Ola

Model: Roadster

Battery: 4 kWh

Price: 120000

Range: [your calculation]

Electric Scooter

Brand: Ather

Model: 450X

Battery: 5 kWh

Price: 150000

Range: [your calculation]

Electric Car

Brand: Tata

Model: Nexon

Battery: 40 kWh

Price: 1500000

Range: [your calculation]

Total Vehicles: 3

For this final question, **you decide the range formula** and implement all the concepts yourself.

Recommended practice order:

Don't solve all 14 at once. Start with: $1 \rightarrow 2 \rightarrow 4 \rightarrow 6 \rightarrow 7 \rightarrow 10 \rightarrow 11 \rightarrow 12 \rightarrow 13 \rightarrow 14$

The last five are especially good for testing whether you actually understand the difference between instance, static, overloading, and overriding.